

**PRINT THE NEAR GREATEST NUMBER OF EACH ELEMENTS IN AN ARRAY:**

```
import java.util.*;
public class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int n= sc.nextInt();
        int arr[] = new int[n];
        for(int i=0;i<n;i++)
        {
            arr[i] = sc.nextInt();
        }
        for(int i=0;i<n;i++)
        {
            int max = -1;
            for(int j=i;j<n;j++)
            {
                if(arr[i]<arr[j])
                {
                    max = arr[j];
                    break;
                }
            }
            System.out.println(arr[i]+" "+max);
        }
    }
}
```

**OUTPUT:**

```
7
5 2 7 1 3 8 4
5 7
2 7
7 8
1 3
3 8
8 -1
4 -1
```

**OPTIMISED:**

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        int arr[] = {5, 2, 7, 1, 3, 8, 4};
        int n = arr.length;
        int result[] = new int[n];
        Stack<Integer> stack = new Stack<>();
```

```

    for (int i = n - 1; i >= 0; i--) {
        while (!stack.isEmpty() && stack.peek() <= arr[i]) {
            stack.pop();
        }
        if (stack.isEmpty()) {
            result[i] = -1;
        } else {
            result[i] = stack.peek();
        }
        stack.push(arr[i]);
    }
    for (int i = 0; i < n; i++) {
        System.out.println(arr[i] + " " + result[i]);
    }
}
}

```

### OUTPUT:

```

5 7
2 7
7 8
1 3
3 8
8 -1
4 -1

```

### PRINT THE FAR GREATEST NUMBER OF EACH ELEMENTS IN AN ARRAY:

```

import java.util.*;
public class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int n= sc.nextInt();
        int arr[] = new int[n];
        for(int i=0;i<n;i++)
        {
            arr[i] = sc.nextInt();
        }
        for(int i=0;i<n;i++)
        {
            int max = -1;
            for(int j=n-1;j>=i+1;j--)
            {
                if(arr[i]<arr[j])
                {
                    max = arr[j];
                    break;
                }
            }
        }
    }
}

```

```

    }
    }
    System.out.println(arr[i]+" "+max);
    }
}

```

### OUTPUT:

```

7
5 2 7 1 3 8 4
5 8
2 4
7 8
1 4
3 4
8 -1
4 -1

```

### OPTIMISED:

```

import java.util.*;
class Main {
    public static void main(String[] args) {
        int arr[] = {5, 2, 7, 1, 3, 8, 4};
        int n = arr.length;
        int suffixMax[] = new int[n];
        suffixMax[n - 1] = arr[n - 1];
        // Build suffix maximum array
        for (int i = n - 2; i >= 0; i--) {
            suffixMax[i] = Math.max(arr[i], suffixMax[i + 1]);
        }
        // Find the rightmost greater element
        for (int i = 0; i < n; i++) {
            if (i == n - 1 || suffixMax[i + 1] <= arr[i])
                System.out.println(arr[i] + " -1");
            else
                System.out.println(arr[i] + " " + suffixMax[i + 1]);
        }
    }
}

```

### OUTPUT:

```

5 8
2 8
7 8
1 8
3 8
8 -1
4 -1

```

### **PRINT THE SUM OF DIGIT NUMBERS IN AN ARRAY**

```
import java.util.*;
public class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        int sum = 0;
        int num = 0;
        for (int i = 0; i < s.length(); i++)
        {
            char ch = s.charAt(i);
            if (Character.isDigit(ch))
            {
                num = num * 10 + (ch - '0');
            }
            else
            {
                sum += num;
                num = 0;
            }
        }
        sum += num;
        System.out.println(sum);
    }
}
```

### **OUTPUT:**

a24b6

30

### **SORT EVEN INDEXED ELEMENTS DESCENDING ODD INDEXED ASCENDING**

```
import java.util.*;
public class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int arr[] = new int[n];
        for(int i=0;i<n;i++)
        {
            arr[i] = sc.nextInt();
        }
        for (int i = 0; i < n - 2; i += 2)
        {
```

```

int max = i;
for (int j = i + 2; j < n; j += 2)
{
    if (arr[j] > arr[max])
    {
        max = j;
    }
}
int temp = arr[i];
arr[i] = arr[max];
arr[max] = temp;
}
for (int i = 1; i < n - 1; i += 2)
{
    int max = i;
    for (int j = i + 2; j < n; j += 2)
    {
        if (arr[j] < arr[max])
        {
            max = j;
        }
    }
    int temp = arr[i];
    arr[i] = arr[max];
    arr[max] = temp;
}
for(int x : arr)
{
    System.out.print(x+" ");
}
}
}

```

### OUTPUT:

```

7
5 2 7 1 3 8 4
7 1 5 2 4 8 3

```

### PRINT THE KTH LARGEST ELEMENT:

```

import java.util.*;
public class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int n= sc.nextInt();
        int arr[] = new int[n];
    }
}

```

```

for(int i=0;i<n;i++)
{
    arr[i] = sc.nextInt();
}
int k = sc.nextInt();
for (int count = 1; count <= k; count++)
{
    int max = Integer.MIN_VALUE;
    int index = -1;
    for(int i=0;i<n;i++)
    {
        if(arr[i]>max)
        {
            max = arr[i];
            index = i;
        }
    }
    if (count == k)
    {
        System.out.println("Kth largest is: "+max);
        break;
    }
    arr[index] = Integer.MIN_VALUE;
}
}
}

```

### **OUTPUT:**

```

7
5 2 7 1 3 8 4
3
Kth largest is: 5

```

### **REMOVE ALL CHARACTERS FROM THE FIRST STRING THAT ARE PRESENT IN THE SECOND STRING**

```

import java.util.*;
public class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        String str1 = sc.nextLine();
        String str2 = sc.nextLine();
        StringBuilder ans = new StringBuilder();
        for (int i = 0; i < str1.length(); i++)
        {

```

```
        char ch = str1.charAt(i);
        boolean found = false;
        for (int j = 0; j < str2.length(); j++)
        {
            if (ch == str2.charAt(j))
            {
                found = true;
                break;
            }
        }
        if (!found)
        {
            ans.append(ch);
        }
    }
    System.out.println(ans);
}
}
```

**OUTPUT:**

programming  
gram  
poin